

2

Managing State in a Simple React App

In the previous chapter, we went over a brief history of web development, **JavaScript**, **ReactJS**, and **React Native**. Even though historical knowledge is not required to write great code, I find it useful. Once we learn why specific library creators encourage some patterns and discourage others, we can write code that is less error-prone and more performant. Ah yes! Writing code! That's why you're here, dear reader, isn't it? Well, I have good news. In this chapter, we will dive into code examples. We will start by looking at the most basic data and state management strategies in React: using state and props. We will then dive into a comparison of stateful and stateless components. Once we have a good understanding of how state works in React applications, we will move on to talking about

hooks. We will finish off this chapter by completing a setup and configuration of our own little app.

Here are the bullet points of what we will cover:

- What is state and how is it different from props?
- What are stateful and stateless components?
- What are hooks and why use them?
- Setting up the example app

By the end of this chapter, you should feel comfortable with React code. We will also set up the base for our application. Even though applications can be very different from one another, this basic setup will remain the same for most of them. Feel free to reuse it for any other project you may want to work on.

Technical requirements

If you are familiar with **ReactJS** but you have not worked with **React Native** yet, you will be able to follow along with this section without any problem.

If you have never read or written any **ReactJS** or **React Native** code, it's important that you learn the basic concepts. Please head over to the official **React Native** documentation at

<https://reactnative.dev/docs/intro-react> and familiarize yourself with key concepts such as **components**, **JSX**, **state**, and **props**.

A minimum requirement for this chapter is knowledge of Git, basic knowledge of **command-line interfaces (CLIs)**, and a working knowledge of JavaScript.

≡ O'REILLY



different from props?

Every **React Native** application is created to display some sort of data. It can be weather data, images, market data, maps...

Using **React Native**, we manage how this data is displayed on our users' screens. **React Native** offers robust tools for styling and animating content. However, in this book, we are concentrating on the raw material of data used in your app.

In order to have a dynamic piece of data, existing **automagically** in sync with our component, we need to declare the list as a component state.

IMPORTANT NOTE

*The most important thing to remember about state is this: state is managed within the component; it is the component **memory**.*

Any changes in state will cause your component and all its children to re-render. This is an expected behavior: if your data changes, you want your UI to change as well. However, multiple component re-renders may cause your app to encounter performance issues.

Let's look at an example to better understand state. We will start off with a very basic component, containing a `<Text>` element and a `<Pressable>` element. `<Pressable>` is the recommended component to use in React Native applications in places where a web developer would use a `<button>` tag:

```
import React from "react";
import { View, Text, Pressable } from "react-native";
export const ManagedButton = () => {
  return (
    <View>
      <Text>this text will display the current status</Text>
      <Pressable onPress="">
        <Text>Press here to check/uncheck</Text>
      </Pressable>
    </View>
  );
};
```


As you can probably observe, dear reader, nothing will happen when the **<Pressable>** component is tapped because we haven't provided an **onPress** function.

We will now add state to this simple component. We will set a checked/unchecked text inside the **<Text>** component, linked to the component state:

```
import React, { useState } from "react";
import { View, Text, Pressable } from "react-native";
export const ManagedButton = () => {
  const [checkedState, setCheckedState] = useState("unchecked");
  return (
    <View>
      <Text>this text will display the current status,
      <Pressable onPress="">
        <Text>Press here to check/uncheck</Text>
      </Pressable>
    </View>
  );
};
```

```
which is: {checkedState}</Text>
```

Testing React Native code is a little more complicated than testing code made to run in browsers (as with JavaScript or ReactJS). Lucky for us, the good people at Expo created an online tool for testing code snippets. It's called Expo Snack and you can use it to test the preceding code at

<https://snack.expo.dev/@p-syche/simplifying-state-management---chapter-2-example-1>.

Let's walk through the changes one by one. We start by adding an import of the **useState** hook from the React library on the first line. Then, inside the component, we set this variable:

```
const [checkedState, setCheckedState] = useState("unchecked");
```

The **useState** hook accepts an array, where the first item is the state value, and the second item is the function that will set the value. If you will not change the state in your component, you can omit the second argument. There is no official rule as per the names of the items in the array, but it is an accepted convention to name the setter function similarly to the state value, but with the **"set"** keyword. Last but not least, the **"unchecked"** string is passed to the **useState** hook. This is the default value of the **useState** hook. If you do not wish to set a default state, you may leave the parentheses empty.

Now that we have the state hook imported and the component state set with the **useState** hook, we can use it in our component. Hence, this line:

```
<Text>this text will display the current status, which is: {checkedState}</Text>
```

The curly braces surrounding the state are part of **JSX**. **JSX** is a syntax extension to **JavaScript**, and it's the syntax used to write all **React** components. "*What does that mean in regular English?*" you ask, dear reader. It means that when writing in **JSX**, you can write any **JavaScript** code, plus you can write additional stuff, such as **component state** wrapped in curly braces. You could look at **JSX** compared to **JavaScript** as if it were a pirate speaking as compared to plain English. All English pirates will understand all English phrases, but a regular Englishman will not understand all pirate phrases. All right, matey? Let's move on then, yo ho ho!

We have the state set up, but our **<Pressable>** component still doesn't do anything, does it? Let's add an **onPress** function, which will set the state. The simplest way to achieve this is to pass the **setCheckedState** function from the **useState** hook right into the **onPress** function:

```
<Pressable onPress={setCheckedState("checked")}>
```

Now, when the **<Pressable>** button is pressed, it will change the state of the component, which in turn will change the text displayed in the **<Text>** component.

There is much more you can achieve with the **useState** hook. You can set it to any value you like, including an object. Every

component can have multiple pieces of state, as many as you'd like, actually! If you would like to look at other examples of how state can be implemented in a React component, I invite you to check the first link from the *Further reading* section.

Let's move on to the second hero of this section: props. Props is a short name for **properties**. Props are **JavaScript** objects just like state; the biggest difference between them is that props are read-only.

IMPORTANT NOTE

The most important thing to remember about props is this: props are immutable (or read-only).

A natural flow of a **ReactJS** or **React Native** app is to have a parent component with some state—in our previous code example, it was the **"checked"/"unchecked"** state. The parent component has children: components with images or text, and so on, to whom we pass the state in the form of a prop. The children can **read** the state: whether the text should be **"checked"** or **"unchecked"**, in this case. But the children will never **change** the state of the text. The **state** of the text can only be changed within the parent component where state was declared. Let's update our code example to include a parent

and a child component, with state set in the parent and passed to the child through props:

```
import React, { useState } from "react";
import { View, Text, Pressable } from "react-native";
const ManagedText = ({checkedState}) => {
  return (
    <Text>this text will display the current status, which is: {checkedState}</Text>
  );
};
export const ParentComponent = () => {
  const [checkedState, setCheckedState] = useState("unchecked");
  return (
    <View>
      <ManagedText checkedState={checkedState} />
      <Pressable onPress={() => setCheckedState("checked")}>
        <Text>Press here to check/uncheck</Text>
      </Pressable>
    </View>
  );
};
```

You can find the preceding code in this Expo Snack:

<https://snack.expo.dev/@p-syche/simplifying-state-management---chapter-2-example-2>.

Let's start with what remained the same as in the previous example. We have our `<ParentComponent>`, which, OK, was named `<ManagedButton>` before. But let's be honest, this com-

ponent didn't change much from the previous version. The only change here is that instead of a `<Text>` component, we see a `<ManagedText>` component, with a mysterious `checkedState` property. This property is passed to the `<ManagedText>` component and then to the `<Text>` component inside it. Pressing the `<Pressable>` component will change the state of `<ParentComponent>`, which will also be reflected in the child component: `<ManagedText>`. I believe, dear reader, that the parent/child nomenclature is quite understandable and doesn't need additional explanations. As per the `checkedState` property, or prop for short, you should know that you can name it whatever you would like; there is no need to set the name of the prop to be the same as its value. You could write something like this, for example:

```
const ManagedText = (fancyComponentStuff) => {  
  return (  
    <Text>this text will display the current status, which is:{fancyComponentStuff}</Text>  
  );  
};  
export const ParentComponent = () => {  
  const [checkedState, setCheckedState] = useState("unchecked");  
  return (  
    <View>  
      <ManagedText fancyComponentStuff={checkedState} />  
      <Pressable onPress={setCheckedState("checked")}>  
        <Text>Press here to check/uncheck</Text>  
      </Pressable>  
    </View>  
  );  
};
```

```
        </Pressable>  
      </View>  
    );  
  };
```

If you're curious to learn more about props and state, you can head over to articles recommended by the official React team. They are listed in the *Further reading* section.

Now that you know what state and props are and how they are different from each other, in the following section, we will look at stateful and stateless components.

What are stateful and stateless components?

Whether you're completely new to the **React** world, or you've been here for a little while, you have probably heard the terms **stateful** and **stateless** components. These terms were especially useful before the introduction of hooks in **ReactJS** v16.8. Don't worry about hooks right now—we'll get to them toward the end of this chapter.

From a high-level perspective, **ReactJS** and **React Native** components are nothing more than **JavaScript** functions. The **React** library adds some specific features to those functions.

One of those features is **state**, a special kind of component memory that we looked at in the previous section.

A React component that can accept state may look like this:

```
class Welcome extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {name: "World"}  
  };  
  render() {  
    return <Text>Hello, {this.state.name}</Text>;  
  }  
}
```

This type of component is also commonly called a “class component” because of the way it needs to be declared. Class components, or stateful components, were first-class citizens until **ReactJS** v16.8. Any developer who needed to do anything with state would use this type of component. Unfortunately, these types of components had many downsides. They used “**lifecycle methods**”—special functions with very specific names, created to be run in a predetermined sequence. There are **componentDidMount()**, **componentWillUnmount()**, **shouldComponentUpdate()**, and a few others. These functions were a lifesaver for many developers facing edge cases. For example, they needed some data to be loaded before the rest of the **component**, or maybe they needed to make sure to clean up some

side effect functions before the **component** unmounted.

Unfortunately, this also meant that their **components** became increasingly complex logically. Trying to understand the flow of the code in a file containing multiple “**lifecycle methods**” is a real challenge. If you would like to learn more about lifecycle methods, please look at the *Further reading* section, where you will find a link to an article in the ReactJS documentation entitled *Adding Lifecycle Methods to a Class*.

Stateful components are also more difficult to test than **stateless components**, plus they compile slower and are bigger after compilation.

Stateless components, also known as **functional components**, are the lightweight brothers of class components. Here’s an example of a stateless component:

```
const Welcome = (props) => {  
  return <Text>Hello, World! </Text>;  
}
```

Comparing the two example **components** shown in the preceding snippet, you should notice a big difference in the number of lines of code needed to write the given **component**. Our simple stateful **component** needed nine lines for what the **functional component** achieved in three!

This means **stateless components** are easier to write from a developer's point of view. They also don't need magical objects such as **constructor** or special **lifecycle methods** such as **componentDidUpdate**. They do, of course, have the great downside of not being able to manage state. So, an ideal **ReactJS** or **React Native** app would include at least one parent, a stateful component, which then would pass props to all kinds of stateless children components. However, there are hardly any ideal apps in the real world. Developers would very often write stateful components and add lifecycle methods to manage when UI updates should and should not happen.

This trend changed with the aforementioned **ReactJS** v16.8 when the concept of hooks was introduced in the **ReactJS** world, which we are going to look at in the next section.

What are hooks and why use them?

As I mentioned before, stateless components are generally easier to write and test. They should be the go-to component of **ReactJS** developers, but they were often overlooked because they could not manage state. At the beginning of 2019, the **ReactJS** team added **hooks** to the library. Hooks add state functionality to stateless components (therefore, it is better to only

use the term **functional components**). One specific hook called **useState** is a function that returns a stateful value and a function to update it. You may recognize it from our previous section about state in React components.

Let's go back to our example of a stateful component, change it to a functional one, and add the **useState** hook, as follows:

```
import React, {useState} from "react";
import {Text} from "react-native";
const Welcome = () => {
  const [name, setName] = useState('World!');
  return <Text>Hello, {name}</Text>;
}
```

Ta-da! It looks so much cleaner than the previous example! We still have a component capable of holding and managing state changes, but it's much shorter than the stateful class component. I also feel this type of component has a very nice logical flow, where we declare the state value and the state setter function on one line.

If you want to see this code in action, you can go to <https://snack.expo.dev/@p-syche/example-of-functional-component-with-usestate>.

This is an **Expo Snack**—an equivalent of code snippets for web development.

Which hooks should you know?

The first hook we spoke about was **useState**, and that one is the absolute first you should familiarize yourself with. The second most used hook is **useEffect**. I also believe this is one of the best-named hooks. You can use it to add all sorts of side effects to your components. "*What is a side effect?*" you may ask, my dear reader. Let's try to grasp this concept using examples: imagine a social media app (much like the app we will be building in this book!). Now, let's imagine you are tasked with adding a likes counter. You have your parent **component** holding the likes button, and a **<Text> component** with a counter. It would look something like this:

```
const LikesParentComponent = () => {
  const getCounterNumberFromApi = someFunctionRetrievingDataFromAPI();
  const [counterNumber, setCounterNumber] = useState(getCounterNumberFromApi)
  return (
    <LikesComponent counterNumber={counterNumber} />
  );
};

const LikesComponent = (counterNumber) => {
  const [likeState, setLikedState] = useState("haven't yet liked");
  return (
```

```
        <View>
          <Text>you {likeState} this post</Text>
          <Pressable onPress={setLikedState("liked")}>
            <Text>Press here to check/uncheck</Text>
          </Pressable>
          <Text>{counterNumber} other people liked this
        </View>
      );
    };
  };
};
```

We are passing **counterNumber** from **<LikesParentComponent>** as a prop. Let's assume this parent component handles retrieving the number of likes from an API using the very nicely named **someFunctionRetrievingDataFromAPI()** function.

This is looking pretty good so far, right? We load our components; they retrieve the likes data from an API and pass it to our **<LikesComponent>**, which displays it nicely. But wait! What happens if the user touches the **<Pressable>** component? We will set **<Text>** to **liked**, but the counter will not go up! We simply cannot leave it like this! This is a classic side effect: a user action requires additional changes in component state. First of all, we cannot change **counterNumber** from within **<LikesComponent>** because, as we learned in the previous section on state and props, props are immutable. What can we do, then? We can use the state setter function from the parent component. This function can be passed as a prop. This means **<LikesParentComponent>** will invoke its child, like this:

post</Text>

```
<LikesComponent counterNumber={counterNumber} setCounterNumber={setCounterNumber} />
```

So far, so good. Now, all we need to do is call this setter function at an appropriate time, which means when the button is pressed in `<LikesComponent>`. This is what it would look like using the `useEffect` hook:

```
const LikesComponent = (counterNumber, setCounterNumber) => {
  const [likeState, setLikedState] = useState("haven't yet liked");
  useEffect(() => {
    if (likeState === "liked") {
      setCounterNumber(counterNumber++)
    }
    else {
      setCounterNumber(counterNumber-1)
    }
  }, [likeState])
  return (
    <View>
      <Text>you {likeState} this post</Text>
      <Pressable onPress={setLikedState("liked")}>
        <Text>Press here to check/uncheck</Text>
      </Pressable>
      <Text>{counterNumber} other people liked this
    </View>
  );
};
```

As you may notice, the **useEffect** hook looks very different from the **useState** hook. Don't worry too much about this. These two hooks are the most used, and you will get used to the way they are conceived and consumed.

The inside of our example **useEffect** hook is a common **if/else** statement checking whether the value of the state equals "**liked**" or not. The most crucial and interesting part of this hook is the array at the very end. This array is called a dependency array. It is used to inform the hook function when it should run. In our case, the **useEffect** hook should run when the value of **likeState** changes.

The **useEffect** hook can be used to update different pieces of the app, to help with data fetching, for user-driven interactions, and so on. This hook is very powerful, but it has a very big risk: it can cause many re-renders when written incorrectly.

THE MOST IMPORTANT THING TO REMEMBER ABOUT USEEFFECT

*Make sure the dependency array of **useEffect** is correct!*

As you may find in the official **ReactJS** documentation, the default behavior for effects is to fire the effect after **every** com-

pleted render. This may often be overkill. In such cases, we can pass an argument to **useEffect**'s dependency array. If we set that, our effect will run *only* if any items in the dependency array change.

There are a few other built-in hooks. You don't have to know them all when starting to write **React Native** code. The two basic hooks—**useState** and **useEffect**—will be enough to get you started. When you get to a point where those two hooks are not enough, you can go back to the **ReactJS** documentation and read about other hooks. You can also write your own custom hooks useful for your particular app.

Now that we know what hooks are and why we use them, let's get started with setting up our sample app!

Setting up the example app

Ah! The moment you've probably been waiting for: actually creating an app!

We will start by preparing our development environment. You will need an **integrated development environment (IDE)** such as VS Code, Sublime Text, Atom, or anything else you may prefer. An IDE is all you need to write **React Native** code. But we also need a way to see what the code renders, don't we?

In the case of web development, we would simply use the browser to see and test our code. However, **React Native** apps cannot be easily tested in a web browser. They can and should be tested on real or simulated devices. In an ideal situation, you would have access to multiple phones, which you would plug into your computer via a USB in order to see your app. Most of us don't have multiple phones, though. That's why we can use phone simulators. There are two major players in the mobile world: Android and Apple. Android simulators are available for virtually any desktop platform thanks to the Android Studio app. Unfortunately, iPhone simulators can be run exclusively on Mac computers.

Setting up simulators can be a daunting task, but don't worry too much! There's **Expo**!

I spoke about **Expo** in the first chapter. If you skipped that part, let me give you a quick rundown: **Expo** is **React Native** development tooling. It makes building, testing, and publishing apps much easier. **Expo** is a wrapper on top of **React Native**, aimed at making the developer experience smoother.

Environment setup

Let's make sure your development environment is ready. As listed on the Expo website, you will need the latest Node, Git,

and Watchman. Links to all of these can be found in Expo's documentation at <https://docs.expo.dev/get-started/installation/>. We will be using Yarn during development, so please make sure you have it installed. You can find detailed instructions here: <https://classic.yarnpkg.com/en/docs/install>. Once you have gone through the links, follow these steps:

1. When you're ready, go ahead and install Expo's CLI tools:

```
$ npm install -global expo-cli
```

2. Verify that the installation was successful by running **expo whoami**. You're not logged in yet, so you will see **Not logged in**. You do not need an account to use Expo. If you want, you can create an account by running **expo register**, or you can log in to an existing account with **expo login**.
3. The next step is to install the Expo Go app on your phone. You can find it in the Android Store at <https://play.google.com/store/apps/details?id=host.exp.exponent> and on the App Store at <https://apps.apple.com/app/expo-go/id982107779>.

Thanks to Expo, it does not matter if you have a Mac computer or a Windows computer and what kind of phone you have. The

Expo Go app will “automagically” work on Android and Apple devices.

4. We’re all set—it’s time to create the app. Go to your terminal and run the following command:

```
$ npx create-expo-app funbook-app
```

5. When prompted about templates, please choose **blank**.

You can choose any name you like for your app. I suggested using “**Funbook**” because it sounds a little like “Facebook” and we’ll be creating a social media app clone. Sticking with the same name as me will probably make it easier to follow along with code examples.

6. After app initialization is successfully run, you can go to your app’s folder by running the following command:

```
$ cd funbook-app
```

7. And run the development server, like so:

```
$ expo start
```

Or, if you're using Yarn, run this command:

```
$ yarn start
```

Expo CLI starts Metro Bundler, which is an HTTP server that compiles the JavaScript code of our app. You should see a QR code that you can now scan using the Expo Go app on your phone. You can run your Funbook app on as many devices as you'd like.

App development can seem a little daunting at first, but don't worry if not everything works perfectly on the first try. There's a big chance you will find the culprit in your terminal window. The terminal output is the best source of information for you.

If you see any errors in the terminal, or you feel a little bit lost, make sure to check the Expo installation documentation:
<https://docs.expo.dev/get-started/create-a-new-app/>.

I set up a public repository that we will use throughout this book. You can find it here:
<https://github.com/PacktPublishing/Simplifying-State-Management-in-React-Native>.

On the **main** branch of this repository, you will find an app that is already set up. Feel free to clone or fork this repository.

Remember that if you want to run this app on your computer, you will still need to install **Expo** tools and other required libraries (**node**, **watchman**, and **yarn**).

App structure

Let us consider which surfaces and components we will need for a simplistic social media app. By “surface,” I mean what would be in web development a “page.” That is a big building block of the app, composed of many components, presented together on the screen.

Our app will definitely need a login surface, a social media feed surface, and a personal profile surface. We will also add a screen containing favorited posts and another one where the user can add their post. We will use fake data for the feed and profile, and a single username and password for logging in. We won’t be implementing a registration flow in order to stay on the simple side of things.

We want to concentrate on data flows, so we will use a free social media UI kit to get the design “out of the way,” so to speak. Here’s a link to the design file we will use:

<https://www.pixeltrue.com/free-ui-kits/social-media-app>.

App root

Our app will consist of at least five surfaces, which means we need to set up navigation to be able to move between those surfaces. The user will start off on the login surface. They will fill in their information and they will be redirected to the social media feed surface.

Obviously, we need a way for our users to move around the app. One of the most used navigation libraries is called **React Navigation**. This is a library created especially for **React Native** applications. It provides three types of navigation out of the box: Drawer navigation, Tab navigation, and Stack navigation. Drawer navigation is when you have a little drawer on the side of your app with links to different places in your app. Tab navigation will display tabs (either bottom or top) with links to different places. Stack navigation works like a stack of cards—each screen is a card having the ability to redirect to any other card. If you would like to know more about this library, you can find a link to the documentation in the *Further reading* section.

There are other navigation libraries out there, but React Navigation is by far the most popular one in the **React Native** community. It is also actively maintained and updated to work with the newest **React Native** versions.

We need to start by adding the library as a dependency to our project. To do that, follow these steps:

1. We can add the library by running the following command:

```
$ yarn add @react-navigation/native
```

2. If you visit the documentation website, you will notice there are different CLI commands for “**Expo** managed projects” and “bare **React Native** projects.” Make sure to follow the instructions for Expo-managed projects. In our case, we need to run the following command:

```
$ expo install react-native-screens react-native-safe-area-context
```

3. We will need to display a login surface first, which will redirect our users to the main app screen. In order to do that, we will use a Stack navigator. Let’s add its dependencies to our project, as listed here at <https://reactnavigation.org/docs/stack-navigator/>:

```
$ yarn add @react-navigation/stack
```

```
$ expo install react-native-gesture-handler
```

The last setup step for the stack navigator is importing the gesture handler library at the very top of our **App.js** file.

The stack navigator will be very useful to manage the login state of our app, but we will also need bottom tab navigation to move between the other screens once the user is logged in. Tab navigation feels very natural for app users. It is visible on all screens and makes using the app easy.

As for now, we will only need to run one command:

```
$ yarn add @react-navigation/bottom-tabs
```

This command adds bottom tab navigation as a dependency to our project so that we will be able to use it later.

You may wonder why we needed to add so many different dependencies separately. This is caused by how the **React Navigation** authors decided to structure their library. They were certain most people will not need every kind of navigation in their **app**, so why should they include it in their app bundle? Every library user can decide which part of **React Navigation** will be useful to them and include only that part.

Let's move on to adding a little bit of structure to our basic app. Every app is built with at least a couple of different surfaces, which in turn are built with components. Our basic social me-

dia clone app will need a login surface and a main surface, visible after login. Since we're creating a social media app, we will go ahead and call the main surface "**Feed**", since it will house the user's newsfeed. As we progress, we will surely add more surfaces, but those two will be a good starting point.

Setting up surfaces

The login surface will need an input field for the username, an input field for the password, and a button to log in. But for now, we will create a dummy component with some text.

We will start by creating the login surface. You may wonder what it means to "create a surface." What I mean by it is that some of the components will be wrappers for entire surfaces of the app. Some people prefer to call them screens, and in web development, you would call them sites or pages. From a coding standpoint, they are components just like any other component. But we decide that, logically, they represent a bigger piece of the app, and we put them in a special folder, called **surfaces**.

Here's our login surface:

```
// ./src/surfaces/Login.js
import React from "react";
import { View, Text } from "react-native";
```

```
export const Login = () => {  
  return (  
    <View>  
      <Text>this will be the login screen</Text>  
    </View>  
  );  
};
```

As you may notice it is, in fact, a dummy component, named **Login**, and placed in the **surfaces** folder.

Using that same logic, we will create a **Feed** surface, which should be displayed after the users log in:

```
// ./src/surfaces/Feed.js  
import React from "react";  
import { View, Text } from "react-native";  
export const Feed = () => {  
  return (  
    <View>  
      <Text>this will be the feed screen</Text>  
    </View>  
  );  
};
```

We have the two basic pieces of the app ready; now we need to put them together. This is where React Navigation comes into play.

Every React Native app needs a root file, just as every website needs an **index.html** file at the root. This root file is usually called **App.js**. This is the **source of truth (SOT)** for displaying anything and everything. You can think of it as a trunk of a tree, with many branches sprouting from it. The branches are different app surfaces in this metaphor. You got that, right? I'm sure you did! You're smart! After all, you *are reading my book*.

Let's set up the parent component to display the correct flow—first, the login screen, and then, the feed:

```
// ./App.js
import 'react-native-gesture-handler';
import React, { useState } from "react";
import { NavigationContainer } from "@react-navigation/native";
import { createStackNavigator } from "@react-navigation/stack";
import { createBottomTabNavigator } from "@react-navigation/bottom-tabs";
import { Login } from "../src/surfaces/Login";
import { Feed } from "../src/surfaces/Feed";
const Stack = createStackNavigator();
const Tab = createBottomTabNavigator();
function Home() {
  return (
    <Tab.Navigator>
      <Tab.Screen name="Feed" component={Feed} />
    </Tab.Navigator>
  );
}
```

```
export default function App() {
  const [userLoggedIn, setIsUserLoggedIn] = useState(true);
  return (
    <NavigationContainer>
      <Stack.Navigator>
        {!userLoggedIn ? (
          <Stack.Screen name="Login" component={Login} />
        ) : (
          <Stack.Screen
            name="Home"
            component={Home}
            options={{ headerShown: false }}
          />
        )}
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

You can find the preceding code in this Expo Snack:

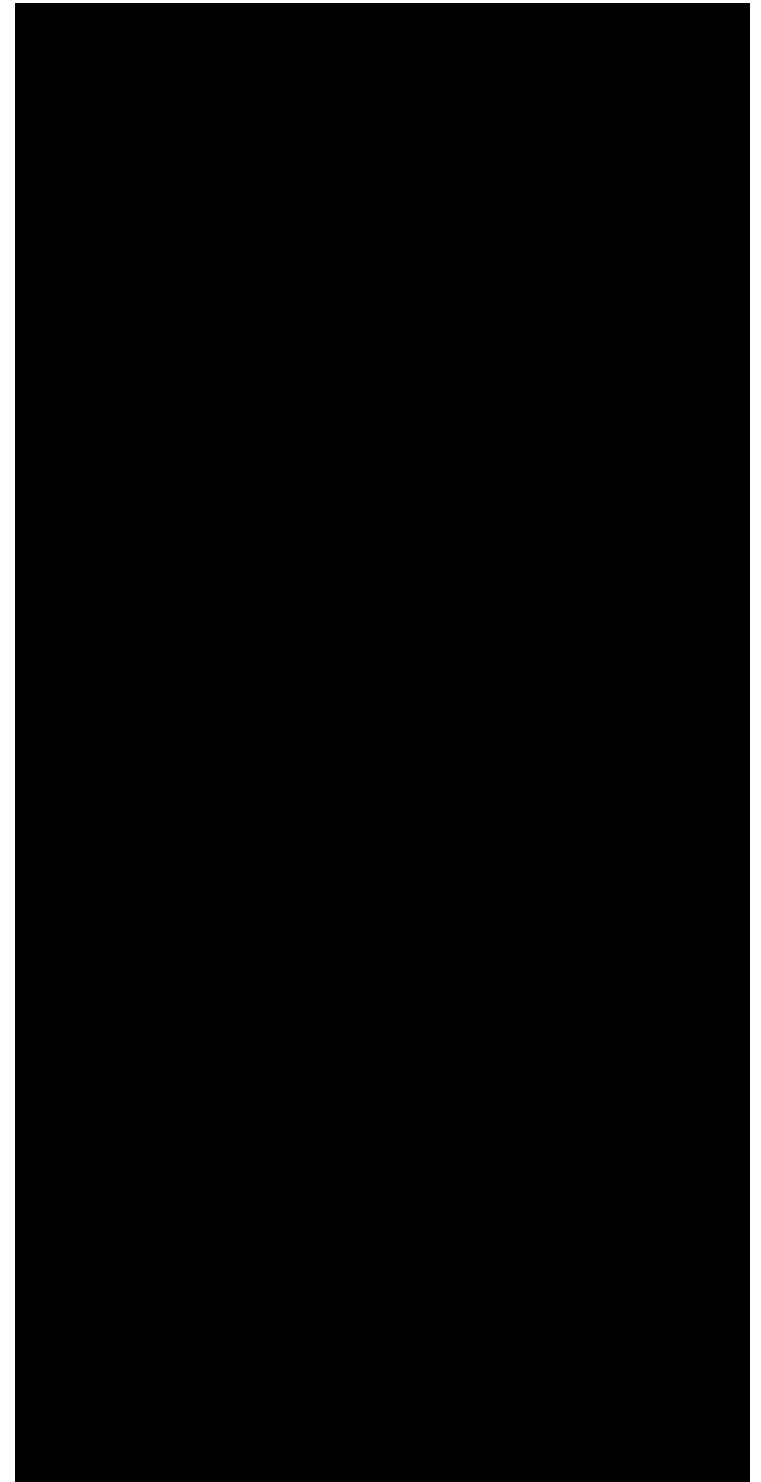
<https://snack.expo.dev/@p-syche/simplifying-state-management---chapter-2-example-3>.

In the preceding code, you will notice we used the `useState` hook. This way, we easily added state to our functional `App` component. We set up our initial state to be **false**—users opening the app for the first time are not supposed to be logged in. When the user logs in, they are redirected to the second “card” in our stack. This “card” is the `Home` component.

This is a wrapper component used to hold the bigger part of our app: all other surfaces besides **Login** with tabbed bottom navigation. As you may notice, the navigators are nested: tabbed navigation is inside the stack navigator. This is a common and useful practice in **React Native** apps. You can read more about nesting navigators in the **React Navigation** documentation here: <https://reactnavigation.org/docs/nesting-navigators>.

And there we go! We have set up an app using **Expo**. We added multiple components representing the future surfaces of the app. We also added and configured the **React Navigation** library. Our app is not very pretty right now, but it should work. You can see it on your phone through the Expo Go app, or in phone simulators on your computer screen.

I set up a public repository on GitHub so that you, dear reader, can more easily follow along with the code snippets and examples presented in this book. You can find the repo here: <https://github.com/PacktPublishing/Simplifying-State-Management-in-React-Native>. Feel free to clone or fork it. The **main** branch includes the basic app setup. Every state management library implementation is on a different branch. We will discuss the details as we move forward. If you decide to use this repository, you will notice the styles from the UI kit are im-



plemented. We will not focus on styling in this book, but it is a nice addition to any app.

Summary

We have done some really good work here! We started out by looking at simple code examples necessary to understand some ReactJS coding concepts such as component state and props, lifecycle methods, and hooks. It is important to understand and internalize the differences between state and props, and stateful and stateless components. A good grasp of those concepts can determine whether your app will run smoothly or not.

After diving into important React concepts and examples, we moved on to actually setting up our app. This is a very exciting moment! We have our foundation, and we are ready to build a real-life social media clone app. In the next chapter, we will get comfortable previewing and debugging our app. We will set up all necessary surfaces, we will add example data, and finally, we will style the app. I can't wait!

Further reading

- React Native’s documentation—example of components with state: <https://reactnative.dev/docs/intro-react#state>.
- State versus Props:

<https://lucybain.com/blog/2016/react-state-vs-pros/>.

<https://github.com/uberVU/react-guide/blob/master/props-vs-state.md>.

- Adding Lifecycle Methods to a Class—ReactJS docs:

<https://reactjs.org/docs/state-and-lifecycle.html#adding-lifecycle-methods-to-a-class>.

- A full blogpost about hooks:

<https://pl.reactjs.org/blog/2019/02/06/react-v16.8.0.html>.

- ReactJS documentation on hooks:

<https://reactjs.org/docs/hooks-reference.html#useeffect>.

- React Navigation documentation:

<https://reactnavigation.org/docs/getting-started/>.

- React Navigation—bottom tab navigation:

<https://reactnavigation.org/docs/tab-based-navigation>.

- React Navigation guide on authentication flow:
[https://reactnavigation.org/docs/auth-flow.](https://reactnavigation.org/docs/auth-flow)
- React Navigation guide on nesting navigators:
[https://reactnavigation.org/docs/nesting-navigators.](https://reactnavigation.org/docs/nesting-navigators)

